LAPORAN PRAKTIKUM W7:
EXCEPTION HANDLING

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:
Rainissa Azizah (251511059)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

Case 1: Exceptions Aren’t Always Errors

Program CountLetters berfungsi untuk membaca sebuah kata dari input *user* lalu menghitung berapa kali setiap huruf muncul. Setiap huruf diubah menjadi angka 0 sampai 25 dengan cara dikurangi 'A', lalu angka tersebut digunakan sebagai indeks array *counts*. Namun, saat program dijalankan dengan input berupa kalimat, misalnya hello world, program akan berhenti dengan pesan error *ArrayIndexOutOfBoundsException*. Hal ini disebabkan oleh spasi yang berkode 32. Ketika 32 dikurangi dengan 65, hasilnya menjadi -33 dan indeks tersebut tidak ada di array yang hanya memiliki indeks 0 sampai 25.

a. Catch kosong

Isi *looping for* yang pertama dimasukkan ke dalam blok *try*, lalu ditambahkan blok *catch* untuk menangkap error *ArrayIndexOutOfBoundsException* dengan bagian isi dikosongkan. Karena baris perintah di dalam *for* sebelumnya hanya ada satu baris, penambahan kurung kurawal kini diperlukan agar strukturnya benar.

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
        // kosong
    }
}
```

Setelah diubah, program tidak lagi berhenti saat menemukan karakter selain huruf. Karakter-karakter tersebut langsung di-*skip* dan tidak ikut dihitung. Sebagai contoh, untuk input "hello world", hasil yang muncul kini hanya menampilkan jumlah huruf D, E, H, L, O, R, dan W saja.

b. Catch menampilkan pesan

Pada perubahan kedua, isi catch diubah agar mencetak pesan beserta exception-nya.

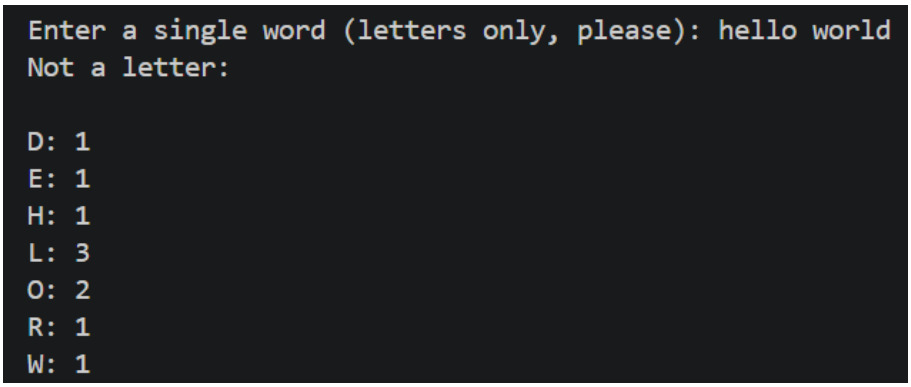
```
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Not a letter: " + e);
}
```

Pesan yang muncul adalah *Not a letter: java.lang.ArrayIndexOutOfBoundsException: Index -33 out of bounds for length 26*. Walaupun informasi tersebut bermanfaat untuk proses *debugging*, pesan itu terlalu panjang untuk situasi yang sebenarnya bukan sebuah kesalahan (*error*). Oleh karena itu, variabel e diubah supaya menampilkan karakter spesifik yang menjadi akar masalahnya secara langsung.

```
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Not a letter: " + word.charAt(i));
}
```

Variabel i masih bisa diakses di dalam blok *catch* karena struktur *catch* tersebut terletak di dalam perulangan *for*. Karena penyesuaian ini, pesan yang ditampilkan menjadi jauh lebih ringkas dan mudah dipahami, yaitu *Not a letter:* yang diikuti langsung oleh karakter penyebab masalahnya.

c. Hasil output



Case 2: Placing Exception Handlers

Program ParseInts berfungsi untuk membaca satu baris teks, memecahnya menjadi beberapa bagian atau token menggunakan *scanner* kedua, lalu mengubah setiap token menjadi bilangan bulat melalui Integer.parseInt sebelum akhirnya menjumlahkannya. Saat diberi *input* angka seperti "10 20 30 40", program berhasil menghitung totalnya menjadi 100. Namun, jika *input* yang dimasukan berupa kalimat biasa seperti *"We have 2 dogs and 1 cat."*, program akan berhenti dan memunculkan error *NumberFormatException* karena kata seperti *"We"* tidak dapat diubah ke dalam bentuk angka.

a. Try-catch membungkus seluruh while

Blok *try* ditempatkan sebelum *looping while*, sementara blok *catch* dipasang tepat setelah badan *looping* berakhir dengan bagian isinya dibiarkan kosong.

```
try {
    while (scanLine.hasNext()) {
        val = Integer.parseInt(scanLine.next());
        sum += val;
```

```
        }
    } catch (NumberFormatException e) {
        // kosong
    }
```

Setelah perubahan, program memang tidak lagi berhenti karena error, tetapi hasil perhitungannya menjadi tidak sesuai harapan. *Input* seperti *"We have 2 dogs and 1 cat."* menghasilkan total 0, sedangkan *"1 fish 2 fish"* hanya menghasilkan 1.

Masalah ini terjadi karena seluruh *looping* disimpan di dalam blok *try*. Akibatnya, ketika ditemukan token non-angka yang pertama, program langsung keluar dari *loop* menuju blok *catch*, sehingga token-token berikutnya tidak sempat dibaca sama sekali.

b. Try-catch dipindahkan ke dalam while

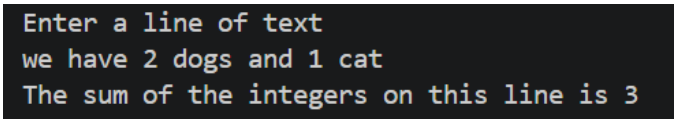
Blok *try-catch* kemudian dipindahkan kembali agar posisinya hanya membungkus bagian dalam *looping while* saja.

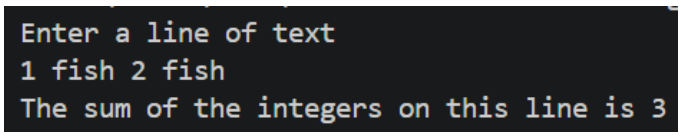
```
while (scanLine.hasNext()) {
    try {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    } catch (NumberFormatException e) {
        // kosong
    }
}
```

Sekarang *exception* ditangkap di setiap putaran *looping*. Kalau ditemukan token yang bukan angka, program akan *skip* putaran tersebut dan langsung melanjutkan *loop* ke token berikutnya.

Dengan perubahan ini, kalimat *input* seperti *"We have 2 dogs and 1 cat."* maupun *"1 fish 2 fish"* sekarang sama-sama menghasilkan total penjumlahan 3. Selain itu, token yang gagal diproses tidak perlu dibaca ulang karena perintah scanLine.next() telah lebih dulu menggeser posisi baca ke token berkutnya sebelum fungsi parseInt dijalankan.

c. Hasil output





Case 3: Throwing Exceptions

Program Factorials memanggil *method* factorial dari *class* MathUtils untuk menghitung nilai faktorial dari angka yang di-*input* oleh *user*. Pada versi awal, *method* tersebut memiliki dua kelemahan utama. Pertama, *method* ini selalu mengembalikan nilai 1 untuk setiap bilangan negatif, padahal faktorial bilangan negatif sebenarnya tidak terdefinisi. Kedua, saat menghitung bilangan yang terlalu besar, hasil perhitungannya berubah menjadi negatif akibat kelebihan kapasitas atau *overflow* yang melampaui batas maksimum tipe data int.

a. Perubahan pada MathUtils

Bagian *header method* tersebut ditambahkan deklarasi *throws IllegalArgumentException*, serta dua kondisi pengecekan di awal *method*. Pengecekan pertama untuk mengantisipasi bilangan negatif, sedangkan pengecekan kedua menangani bilangan di atas angka 16. Kedua pengecekan ini berfungsi untuk melempar *exception IllegalArgumentException* dengan pesan yang berbeda agar penyebab masalahnya diketahui dengan jelas.

```
public class MathUtils {
    public static int factorial(int n) throws IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException("Factorial is not defined for
negative numbers:    " + n);
        }
        if (n > 16) {
            throw  new  IllegalArgumentException("Argument  is  too  large
(maximum is 16): " + n);
        }
        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

Proses pengecekan diletakkan di awal *method* supaya perhitungan tidak dijalankan saat argumen yang diberikan tidak valid. Selain itu, perintah *throw* berfungsi untuk menghentikan *method* saat itu juga sekaligus mengirimkan objek *exception* kepada bagian yang memanggilnya. Setelah MathUtils diubah, saat *user input* angka -3, program langsung berhenti dengan memunculkan pesan error *IllegalArgumentException*. Hal ini terjadi karena *exception* tersebut

dilempar dari dalam *method factorial*, lalu merambat naik ke fungsi *main*, tetapi tidak ada blok *catch* yang menangkapnya sehingga program terpaksa mengakhiri eksekusinya.

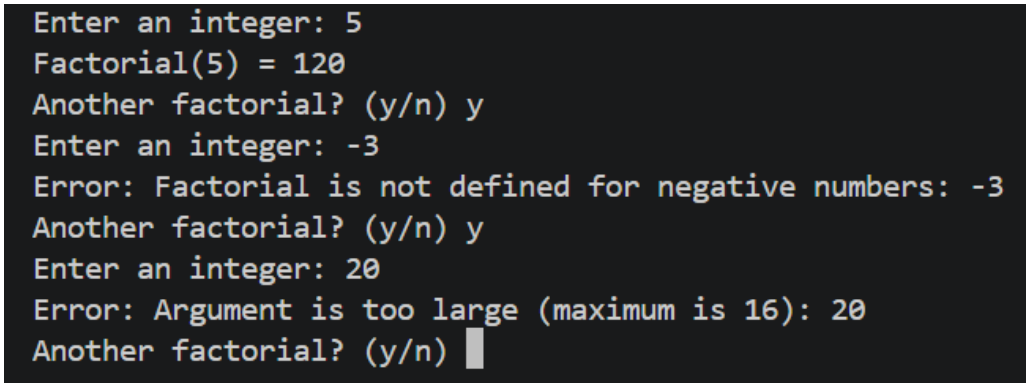
b. Perubahan pada Factorials

Blok *try-catch* ditambahkan ke dalam *looping while* di dalam fungsi *main*, dengan posisi yang hanya membungkus baris kode pemanggil *method factorial*.

```
while (keepGoing.equals("y") || keepGoing.equals("Y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();
    try {
        System.out.println("Factorial(" + val + ") = " +
   MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```

Penggunaan *method* e.getMessage() berfungsi untuk mengambil teks pesan yang disertakan saat perintah *throw* dijalankan, sehingga pesan error yang muncul selalu sesuai dengan masalahnya. Pertanyaan *"Another factorial?"* sengaja disimpan di luar blok *try-catch* supaya tetap dapat ditampilkan meskipun terjadi kesalahan, sehingga *looping* dapat terus berlanjut.

c. Hasil output



Angka yang valid sekarang berhasil menampilkan nilai faktorial dengan tepat. Sementara itu, *input* berupa bilangan negatif maupun angka di atas 16 akan menampilkan pesan error yang berbeda dan program tetap dapat berjalan hingga *user* memilih untuk menghentikannya.

Lesson Learn

Exception tidak selamanya berarti kesalahan. Misalnya dalam *Case 1*, kemunculan karakter selain huruf pada *input* data merupakan hal yang wajar, asalkan karakter tersebut tidak ikut dihitung. Oleh karena itu, cukup menangkapnya menggunakan blok *catch* tanpa harus ribet menyusun validasi atau pengecekan manual.

Sementara itu, pada *Case 2* dan *Case 3*, penempatan blok *try-catch* sangat menentukan bagaimana alur program akan dijalankan. Jika blok tersebut membungkus semua *looping*, satu kejadian *exception* saja sudah cukup untuk menghentikan seluruh proses. Sebaliknya, jika blok *try-catch* disimpan di dalam lingkup *loop*, program hanya akan mengabaikan *loop* yang bermasalah saja dan proses utamanya tetap berjalan secara normal.

Selain itu, jika suatu *method* tidak bisa menghasilkan nilai yang valid, seperti fungsi faktorial yang menerima *input* bilangan negative, menggunakan perintah *throw* untuk memunculkan *exception* lebih disarankan daripada memaksakan pengembalian nilai sembarang yang berpotensi menyesatkan hasil akhir program.

Link Repo

https://github.com/rainissa/PBO-Praktik.git